

- **Aim:** Hands on Solidity Programming Assignments for creating Smart Contracts

- **Theory:**

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
 - **public:** available both inside and outside the contract.
 - **private:** only accessible within the same contract.
 - **internal:** accessible within the contract and its child contracts.

- o external: can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

? **Implementation:**

- Tutorial no. 1 – Compile the code

SOLIDITY COMPILER

COMPILER +

0.8.18+commit.87f61d96

☐ Include nightly builds

☐ Auto compile

☐ Hide warnings

Advanced Configurations >

Compile introduction.sol

Compile and Run script

CONTRACT

Counter (introduction.sol)

Run Remix Analysis

Run SolidityScan

Publish on IPFS

Publish on Swarm

Compilation Details

introduction.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) {  2459 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public {  infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public {  infinite gas
19        count -= 1;
20    }
21 }
```

Explain contract

0 ☐ Lister

Welcome to Remix 0.1.0

- Tutorial no. 1 – Deploy the contract

The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel displays the 'Remix VM' environment with 'Account 1' (0x5B3...eddC4) having a balance of 99.999 ETH. The 'Interact with a deployed contract' section shows the 'Counter' contract at address 0xd91...39138. The 'Functions' section is empty, and the 'Low level interaction' section is also empty. The 'Transactions recorder 2' panel at the bottom shows a list of transactions. The main editor displays the Solidity code for the 'Counter' contract:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }

```

The 'Explain contract' panel at the bottom right shows the contract's ABI and a list of transactions. The first transaction is a constructor call from 0xb83...eddC4 to Counter.(constructor) with a value of 0 wei and data 0xb08...20033. The second transaction is a call from 0x5B3...eddC4 to Counter.get() with a value of 0 wei and data 0xd4...ce63c.

- Tutorial no. 1 – get

The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel displays the 'Remix VM' environment with 'Account 1' (0x5B3...eddC4) having a balance of 99.999 ETH. The 'Interact with a deployed contract' section shows the 'Counter' contract at address 0xd91...39138. The 'Functions' section is empty, and the 'Low level interaction' section is also empty. The 'Transactions recorder 2' panel at the bottom shows a list of transactions. The main editor displays the Solidity code for the 'Counter' contract:

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public { infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }

```

The 'Explain contract' panel at the bottom right shows the contract's ABI and a list of transactions. The first transaction is a constructor call from 0xb83...eddC4 to Counter.(constructor) with a value of 0 wei and data 0xb08...20033. The second transaction is a call from 0x5B3...eddC4 to Counter.get() with a value of 0 wei and data 0xd4...ce63c.

- Tutorial no. 1 – Increment

Low level interaction +

inc

Value: 0 wei

Gas limit: auto 0

Transact

Balance: 0 ETH

Explain contract AI copilot

0 ☐ Listen on all transactions

If the transaction failed for not having enough gas, try increasing the gas limit gently..

Error occurred: revert.

revert The transaction has been reverted to the initial state.

Note: The called function should be payable if you send value and the value you send should be less than your c

If the transaction failed for not having enough gas, try increasing the gas limit gently.

If the transaction failed for not having enough gas, try increasing the gas limit gently.

[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei
data: 0x371...303c0 logs: 0 hash: 0xeaf...98f8c **Debug**

- Tutorial no. 1 – Decrement

Account 1 99.999 ETH
0x5B3...eddC4

Select a function to interact with...

Low level interaction +

dec

Value: 0 wei

Gas limit: auto 0

Transact

Balance: 0 ETH

Explain contract AI copilot

0 ☐ Listen on all transactions

revert The transaction has been reverted to the initial state.

Note: The called function should be payable if you send value and the value you send should be less than your c

If the transaction failed for not having enough gas, try increasing the gas limit gently.

If the transaction failed for not having enough gas, try increasing the gas limit gently.

[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei
data: 0x371...303c0 logs: 0 hash: 0xeaf...98f8c **Debug**

[vm] from: 0x583...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei
data: 0xb3b...cfa82 logs: 0 hash: 0xf1a...8fc3b **Debug**

- Tutorial no. 2

The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel shows 'Remix VM' and 'Osaka' network. 'Account 1' with address '0x5B3...eddC4' has a balance of '99.999 ETH'. The 'Functions' panel shows a function named 'name' with a return type of 'string'. The 'Call' button is visible. The main editor shows the Solidity code for 'MyContract'.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract MyContract {
5     string public name = "Saloni";
6 }

```

The 'Explain contract' panel shows a transaction log with two entries:

- [vm] from: 0x5B3...eddC4 to: MyContract.(constructor) value: 0 wei** (hash: 0xe88...2289e)
- [call] from: 0x5B38Da6a701c56854dCfcB03Fc8875f56beddC4 to: MyContract.name()** (hash: 0x06f...dde03)

- Tutorial no. 3

The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel shows 'Remix VM' and 'Osaka' network. 'Account 1' with address '0x5B3...eddC4' has a balance of '99.999 ETH'. The 'Functions' panel shows a function named 'newAddr' with a return type of 'address'. The 'Call' button is visible. The main editor shows the Solidity code for 'Primitives'.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Primitives {
5     bool public boo = true;
6
7     /*
8      * uint stands for unsigned integer, meaning non negative integers
9      * different sizes are available
10      * uint8 ranges from 0 to 2 ** 8 - 1
11      * uint16 ranges from 0 to 2 ** 16 - 1
12      * ...
13      * uint256 ranges from 0 to 2 ** 256 - 1
14      */
15     uint8 public u8 = 1;
16     uint256 public u256 = 456;
17     uint public u = 123;
18
19     /*
20      * Negative numbers are allowed for int types.
21      * Like uint, different ranges are available from int8 to int256
22      */
23 }

```

The 'Explain contract' panel shows a transaction log with one entry:

- [vm] from: 0x5B3...eddC4 to: Primitives.(constructor) value: 0 wei** (hash: 0x24f...69678)

- Tutorial no. 4

The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel shows 'Remix VM' selected with 'Osaka' as the fork. 'Account 1' is listed with a balance of 99.999 ETH. The 'Functions' section shows a 'text' function selected, with its output '0:string: Saloni' displayed. A 'Call' button is at the bottom of this section. The main editor displays the Solidity code for 'variables.sol':

```

5 // State variables are stored on the blockchain.
6 string public text = "Saloni";
7 uint public num = 123;
8 uint public blockNumber;
9
10 function doSomething() public { 22334 gas
11     // Local variables are not saved to the blockchain.
12     uint i = 456;
13
14     // Here are some global variables
15     uint timestamp = block.timestamp;
16     address sender = msg.sender;
17
18     blockNumber = block.number;
19 }
20

```

Below the code, the 'Explain contract' panel shows a transaction log with a 'CALL' entry: '[call] from: 0x5B38Da6a701c568545dCfc803FcB875f56beddC4 to: Variables.text() data: 0x1f'.

- Tutorial no. 5

The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel is the same as in the previous screenshot. The 'Functions' section now shows a 'get_b' function selected, with its output '0:bool: true' displayed. A 'Call' button is at the bottom. The main editor displays the Solidity code for 'SimpleStorage.sol':

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7
8     bool public b = true;
9
10    // You need to send a transaction to write to a state variable.
11    function set(uint _num) public { 22536 gas
12        num = _num;
13    }
14
15    // You can read from a state variable without sending a transaction.
16    function get() public view returns (uint) { 2475 gas
17        return num;
18    }
19 }

```

Below the code, the 'Explain contract' panel shows a transaction log with a 'CALL' entry: '[call] from: 0x5B38Da6a701c568545dCfc803FcB875f56beddC4 to: SimpleStorage.get_b() data: 0xf3d...33cf4'.

• Tutorial no. 6

LEARNETH

Tutorials list

5.2 Functions - View and Pure
6 / 19

5. Using inline assembly that contains certain opcodes.*
From the [Solidity documentation](#).

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ **Assignment**

Create a function called `addTox2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer **Show answer**

Next

Well done! No errors.

```
3
4 contract ViewAndPure {
5   uint public x = 1;
6
7   // Promise not to modify the state.
8   function addTox(uint y) public view returns (uint) {
9     return x + y;
10  }
11
12  // Promise not to modify or read from the state.
13  function add(uint i, uint j) public pure returns (uint) {
14    return i + j;
15  }
16
17  function addTox2(uint y) public {
18    x = x + y;
19  }
20 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

hash: 0xe81...1e110

[vm] from: 0x583...eddC4 to: ViewAndPure.addTox2(uint256) 0x0A0...42853 value: 0 wei data: 0xfe3...00005 logs: 0 hash: 0xd9b...b5b3d **Debug**

• Tutorial no. 7

REMIX 2.5.7

Welcome to the new home of Remix. Bookmark [app.remix.live](#) so you always land here. [Still need your files from remix.ethereum.org?](#)

Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.3 Functions - Modifiers and Constructors
7 / 19

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

[Watch a video tutorial on Function Modifiers.](#)

★ **Assignment**

1. Create a new function, `increaseX`, in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.

2. Make sure that `x` can only be increased.

3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer **Show answer**

Next

Well done! No errors.

```
44
45   locked = false;
46 }
47
48 // Modifier to increase x.
49 modifier increaseXBy(uint _i) {
50   x += _i;
51   _;
52 }
53
54 // The function body is intentionally empty.
55 function increaseX(uint _i) public increaseXBy(_i) {
56 }
57
58 function decrement(uint i) public noReentrancy {
59   x -= i;
60
61   if (i > 1) {
62     decrement(i - 1);
63   }
64 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

hash: 0x285...50e2c

[vm] from: 0x583...eddC4 to: FunctionModifier.increaseX(uint256) 0xcd6...990f9 value: 0 wei data: 0x975...00005 logs: 0 hash: 0x217...69076 **Debug**

• Tutorial no. 8

LEARNETH 2.5.7 Learneth Solidity Beginner Course

Tutorials list 5.4 Functions - Inputs and Outputs 8 / 19

There are a few restrictions and best practices for the input and output parameters of contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

[Watch a video tutorial on Function Outputs.](#)

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer **Show answer**

Next

Well done! No errors.

```

74
75 function arrayOutput() public view returns (uint[] memory) {
76     return arr;
77 }
78
79 // Assignment
80 function returnTwo() public
81     pure
82     returns (
83         int x,
84         bool b
85     )
86 {
87     x = -2;
88     b = true;
89 }
90
91

```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

hash: 0x285...50e2c

[vm] from: 0x583...eddC4 to: FunctionModifier.increaseX(uint256) 0xC06...990F9 value: 0 wei data: 0x975...00005 logs: 0 hash: 0x217...69076

• Tutorial no. 9

LEARNETH 2.5.7 Learneth Solidity Beginner Course

Tutorials list 6. Visibility 9 / 19

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

[Watch a video tutorial on Visibility.](#)

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer **Show answer**

Next

Well done! No errors.

```

35
36 function testInternalFunc() public
37     pure
38     override
39     returns (string memory)
40 {
41     return internalFunc();
42 }
43
44
45 function testInternalVar() public
46     view
47     returns (string memory, string memory)
48 {
49     return (internalVar, publicVar);
50 }
51
52

```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

hash: 0x285...50e2c

[vm] from: 0x583...eddC4 to: FunctionModifier.increaseX(uint256) 0xC06...990F9 value: 0 wei data: 0x975...00005 logs: 0 hash: 0x217...69076

• Tutorial no. 10

LEARNETH 2.5.7 Learneth Solidity Beginner Course

Tutorials list 7.1 Control Flow - If/Else 10 / 19

the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer **Show answer**

Next

Well done! No errors.

```

5 function foo(uint _x) public pure returns (uint) {
6     if (_x < 10) {
7         return 0;
8     } else if (_x < 20) {
9         return 1;
10    } else {
11        return 2;
12    }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16     return _x < 10 ? 1 : 2;
17 }
18
19 function evenCheck(uint _x) public pure returns (bool) {
20     return _x % 2 == 0 ? true : false;
21 }
22

```

Explain contract

0 ☐ Listen on all transactions

hash: 0x285...50e2c

[vm] from: 0x5B3...eddC4 to: FunctionModifier.increaseX(uint256) @0x06...990f9 value: 0 wei data: 0x975...00005 logs: 0 hash: 0x217...69876

Debug

• Tutorial no. 11

LEARNETH 2.5.7 Learneth Solidity Beginner Course

Tutorials list 7.2 Control Flow - Loops 11 / 19

this contract, the `break` statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer **Show answer**

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     uint public count;
6
7     function loop() public {
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10             if (i == 5) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18             count++;
19         }
20     }
21 }

```

Explain contract

0 ☐ Listen on all transactions

Type the library name to see available commands.

- Tutorial no. 12

REMIX 2.6.0

Learneth Solidity Beginner Course

AI Code

LEARNETH

Tutorials list Syllabus

8.1 Data Structures - Arrays 12 / 19

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

Did you know? You can verify your contract using the Sourcify plugin.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8     // Fixed sized array, all elements initialize to 0
9     uint[10] public myFixedSizeArr;
10    uint[3] public arr3 = [0, 1, 2];
11
12    function get(uint i) public view returns (uint) {
13        return arr[i];
14    }
15
16    // Solidity can return the entire array.
17    // But this function should be avoided for
18    // arrays that can grow indefinitely in length.
19    function getArr() public view returns (uint[3] memory) {
20        return arr3;
21    }
22 }
```

Explain contract

0 ☐ Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

- Tutorial no. 13

REMIX 2.6.0

Learneth Solidity Beginner Course

AI Code

LEARNETH

Tutorials list Syllabus

8.2 Data Structures - Mappings 13 / 19

[Watch a video tutorial on Mappings.](#)

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo

Did you know? You can verify your contract using the Sourcify plugin.

```

26 // Nested mapping (mapping from address to another mapping)
27 mapping(address => mapping(uint => bool)) public nested;
28
29 function get(address _addr1, uint _i) public view returns (bool) {
30     // You can get values from a nested mapping
31     // even when it is not initialized
32     return nested[_addr1][_i];
33 }
34
35 function set(
36     address _addr1,
37     uint _i,
38     bool _boo
39 ) public {
40     nested[_addr1][_i] = _boo;
41 }
42
43 function remove(address _addr1, uint _i) public {
44     delete nested[_addr1][_i];
45 }
46 }
```

Explain contract

0 ☐ Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

- Tutorial no. 14

LEARNETH 2.6.0

Learneth Solidity Beginner Course

AI Code

✓ Compiled

answer.sol arrays.sol arrays_answer.sol mappings.sol mappings_answer.sol

Create a DApp with your contracts Start now

32 function getSize(uint _index) public view returns (Size) {
33 Todo storage todo = todos[_index];
34 return (todo.text, todo.completed);
35 }
36
37 // update text
38 function update(uint _index, string memory _text) public { infinite gas
39 Todo storage todo = todos[_index];
40 todo.text = _text;
41 }
42
43 // update completed
44 function toggleCompleted(uint _index) public { 28995 gas
45 Todo storage todo = todos[_index];
46 todo.completed = !todo.completed;
47 }
48
49 function remove(uint _index) public { infinite gas
50 delete todos[_index];
51 }

0 ☐ Listen on all transactions

Type the library name to see available commands.

Well done! No errors.

- Tutorial no. 15

LEARNETH

Compiled

answer.sol mappings.sol mappings_answer.sol structs.sol structs_answer.sol

Create a DApp with your contracts Start now

28
29 function getSize() public view returns (Size) { 2633 gas
30 return sizes;
31 }
32
33 // Update status by passing uint into input
34 function set(Status _status) public { undefined gas
35 status = _status;
36 }
37
38 // You can update to a specific enum like this
39 function cancel() public { 24494 gas
40 status = Status.Canceled;
41 }
42
43 // delete resets the enum to its first value, 0
44 function reset() public { 24383 gas
45 delete status;
46 }
47 }

0 ☐ Listen on all transactions

Type the library name to see available commands.

Well done! No errors.

- Tutorial no. 16

REMIX 2.6.0

Learneth Solidity Beginner Course

AI Code

LEARNETH

Tutorials list

9. Data Locations 16 / 19

the data location `memory` inside the function `f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.

3. Create a new struct `myMemStruct3` with the data location `memory` inside the function `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.

4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer

Show answer

Well done! No errors.

Scam Alert Initialize as git repo

Did you know? You can verify your contract using the Sourcify plugin.

```

28 function _f( ) undefined gas
29     uint[] storage _arr,
30     mapping(uint => address) storage _map,
31     MyStruct storage _myStruct
32 } internal {
33     // do something with storage variables
34 }
35
36 // You can return memory variables
37 function g(uint[] memory _arr) public returns (uint[] memory) { infinite gas
38     // do something with memory array
39     _arr[0] = 1;
40 }
41
42 function h(uint[] calldata _arr) external { 468 gas
43     // do something with calldata array
44     _arr[0] = 1;
45 }
46 }
47

```

Explain contract

0 ☐ Listen on all transactions

Type the library name to see available commands.

- Tutorial no. 17

LEARNETH

Tutorials list

10.1 Transactions - Ether and Wei 17 / 19

1,000,000,000,000,000,000 (10¹⁸) wei (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 gwei.

2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 gwei and 10⁹.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

sol enums_answer.sol dataLocations.sol 2 dataLocations_ar

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }

```

Explain contract

0 ☐ Listen on all transactions

Type the library name to see available commands.

- Tutorial no. 18

REMIX 2.6.0 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

10.2 Transactions - Gas and Gas Price 18 / 19

Learn more about *gas* on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer

Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? You can verify your contract using the Sourcify plugin.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }

```

Create a DApp with your contracts Start now

Explain contract

0 ☐ Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

- Tutorial no. 19

LEARNETH

Tutorials list

10.3 Transactions - Sending Ether 19 / 19

1. Create a contract called `Charity`.

2. Add a public state variable called `owner` of the type `address`.

3. Create a donate function that is public and payable without any parameters or function code.

4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer

Show answer

Next

Well done! No errors.

Alert Initialize as git repo Did you know? You can verify your contract using the Sourcify plugin.

```

49 require(sent, "Failed to send Ether");
50 }
51 }
52
53 contract Charity {
54     address public owner;
55
56     constructor() {
57         owner = msg.sender;
58     }
59
60     function donate() public payable {}
61
62     function withdraw() public {
63         uint amount = address(this).balance;
64
65         (bool sent, bytes memory data) = owner.call{value: amount}("");
66         require(sent, "Failed to send Ether");
67     }
68 }

```

Create a DApp with your contracts Start now

Explain contract

0 ☐ Listen on all transactions Filter with transaction hash or address

Type the library name to see available commands.

□ **Conclusion:**

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.